## DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Метод DBSCAN относится к алгоритмам кластеризации, основанным на анализе плотности распределения данных. В отличие от многих классических подходов, он не требует заранее задавать количество кластеров и способен выявлять выбросы.

### Суть метода

Основная идея алгоритма заключается в том, что кластеры представляют собой области с высокой плотностью точек, отделённые друг от друга областями с низкой плотностью.

Алгоритм определяет, какие точки образуют плотные области, и объединяет их в кластеры. Точки, которые не принадлежат ни одной плотной области, считаются шумом.

Таким образом, структура данных определяется не расстоянием до центров, а плотностью расположения объектов.

### Основные понятия

#### eps-окрестность

Для каждой точки рассматривается область радиуса ε\varepsilonε. Все точки внутри этой области считаются соседями.

#### min_samples

Минимальное количество точек в eps-окрестности, необходимое для признания точки "плотной".

### Типы точек

1. **Ядровая точка (core point)** — имеет не менее min_samples соседей
2. **Граничная точка (border point)** — находится рядом с ядровой, но сама не удовлетворяет условию плотности
3. **Шум (noise)** — не принадлежит ни одному кластеру

### Принцип работы

Алгоритм проходит по всем точкам и выполняет следующие действия:

1. Если точка является ядровой, начинается формирование нового кластера
2. Все точки в её eps-окрестности добавляются в кластер
3. Для каждой из этих точек проверяется, является ли она ядровой

4. Процесс продолжается, пока кластер не перестанет расширяться

Таким образом формируются кластеры произвольной формы.

**Математическая формализация**

Окрестность точки определяется как:

$$N_\varepsilon(p) = \{q \mid dist(p, q) \leq \varepsilon\}$$

Точка считается ядровой, если:

$$|N_\varepsilon(p)| \geq min_samples$$

**Реализация на Python**

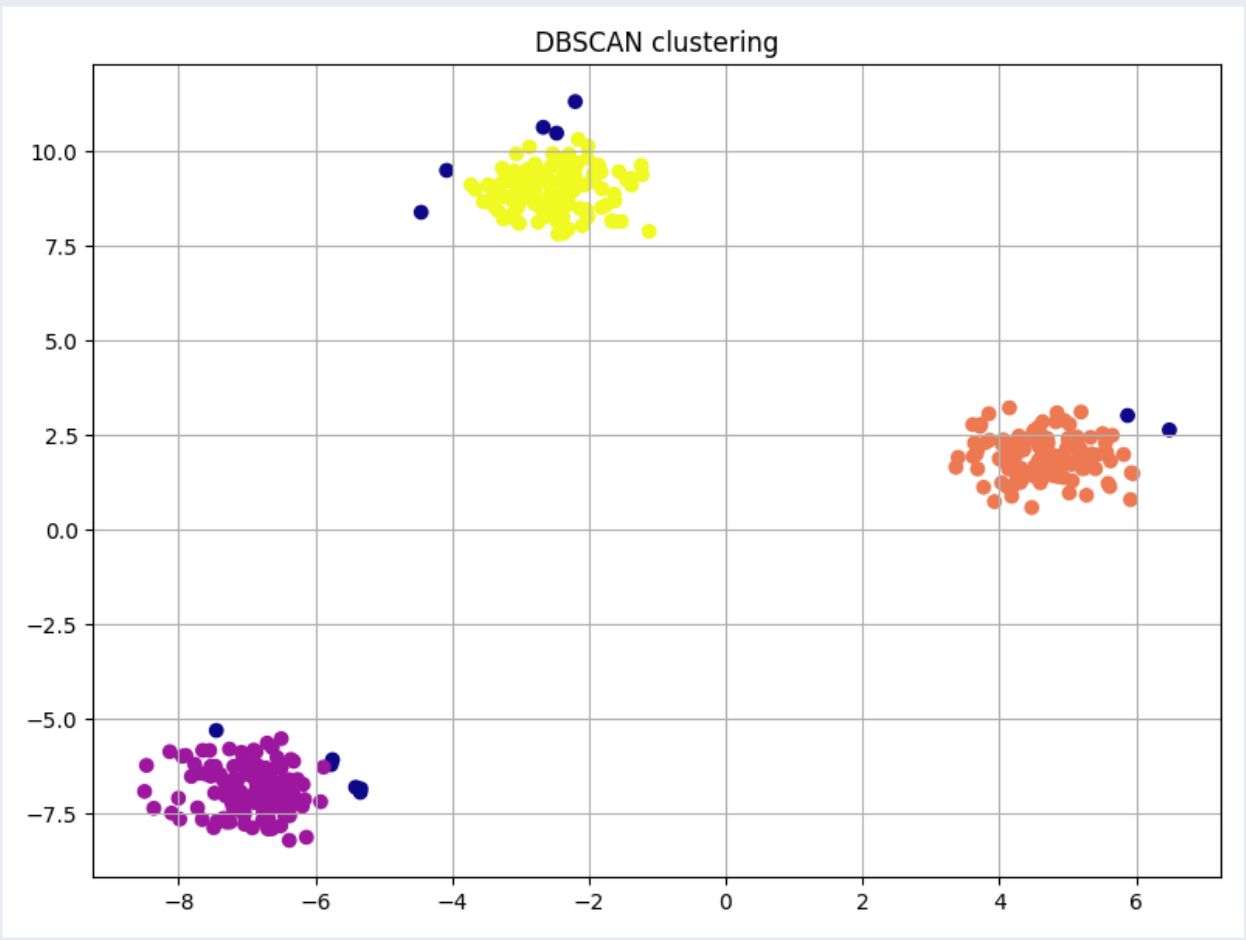
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_blobs

# Генерация данных
X, _ = make_blobs(
    n_samples=400,
    centers=3,
    cluster_std=0.6,
    random_state=42
)

# Создание модели
model = DBSCAN(
    eps=0.5,
    min_samples=5
)

# Обучение
labels = model.fit_predict(X)

# Визуализация
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='plasma')
plt.title("DBSCAN clustering")
plt.grid(True)
plt.show()
```




**Пояснение к примеру**

В данном примере создаётся набор точек с тремя кластерами. Затем применяется алгоритм DBSCAN, который автоматически определяет структуру данных.

Особенность результата заключается в том, что часть точек может быть помечена как шум (обычно метка -1). Это позволяет выявлять выбросы, что является важным преимуществом метода.

**Основные параметры**

| Параметр | Описание | Влияние |
|---|---|---|
| eps | радиус окрестности | определяет плотность кластера |
| min_samples | минимальное число соседей | влияет на чувствительность к шуму |
| metric | метрика расстояния | определяет способ измерения расстояния |

**Достоинства метода**

1. Не требует задания числа кластеров

2. Умеет находить кластеры произвольной формы
3. Обнаруживает выбросы
4. Устойчив к шуму

**Недостатки метода**

1. Чувствителен к параметрам eps и min_samples
2. Плохо работает при неоднородной плотности
3. Сложно подобрать параметры без анализа данных
4. Может объединять близкие кластеры

**Задачи для проверки понимания**

**Задача 1. Влияние eps**

Вопрос: Что произойдёт, если увеличить значение eps?

Ответ:
При увеличении eps радиус окрестности увеличивается, в результате чего больше точек начинают считаться соседями. Это приводит к объединению кластеров и уменьшению их общего количества. В предельном случае все точки могут попасть в один кластер.

**Задача 2. Влияние min_samples**

Вопрос: Как изменится результат при увеличении min_samples?

Ответ:
При увеличении min_samples требования к плотности возрастают. В результате часть точек перестаёт считаться ядровыми и может быть отнесена к шуму. Это приводит к уменьшению числа кластеров и увеличению количества выбросов.

**Задача 3. Обнаружение шума**

Вопрос: Почему DBSCAN способен находить выбросы?

Ответ:
Алгоритм явно выделяет точки, которые не принадлежат ни одной плотной области. Такие точки не имеют достаточного числа соседей и не входят в eps-окрестность ядровых точек, поэтому классифицируются как шум.

**Задача 4. Практический подбор параметров**

Условие:
Дан набор данных, содержащий неизвестное количество кластеров.
Требуется подобрать eps так, чтобы получить наилучшее разбиение.

Решение:

```
import numpy as np
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_blobs
from sklearn.metrics import silhouette_score

X, _ = make_blobs(
    n_samples=500,
    centers=4,
    cluster_std=0.7,
    random_state=42
)

best_eps = None
best_score = -1
best_clusters = None

for eps in np.linspace(0.2, 1.2, 10):
    model = DBSCAN(eps=eps, min_samples=5)
    labels = model.fit_predict(X)

    unique_labels = set(labels)

    if -1 in unique_labels:
        unique_labels.remove(-1)

    n_clusters = len(unique_labels)

    if n_clusters > 1:
        mask = labels != -1
        score = silhouette_score(X[mask], labels[mask])

        if score > best_score:
            best_eps = eps
            best_score = score
            best_clusters = n_clusters

print(f"Лучший eps: {best_eps}")
print(f"Количество кластеров: {best_clusters}")
print(f"Silhouette Score: {best_score:.3f}")
```

Пояснение:
В данном примере перебираются различные значения eps. Для каждого варианта оценивается качество кластеризации с помощью метрики силуэта.

Шумовые точки исключаются из оценки, так как они не принадлежат ни одному кластеру.

**Итог**

Метод DBSCAN представляет собой мощный инструмент кластеризации, позволяющий выявлять структуру данных на основе плотности. Его ключевое преимущество заключается в способности находить кластеры произвольной формы и автоматически определять выбросы. Однако для получения качественного результата требуется внимательный подбор параметров и анализ распределения данных.